

mi_cuenta_trainer.js — Fundamentos

Explicación detallada del perfil del entrenador (versión refactorizada)

Es la pantalla de **perfil** del entrenador — la segunda (y última) pestaña de su barra de navegación. Es mayormente de **solo lectura**: muestra los datos del usuario logueado y ofrece cerrar sesión, sin formularios ni edición. Esta versión ya incorpora el subcomponente ProfileDataCard, la confirmación con Alert al cerrar sesión, y dos correcciones aplicadas sobre la implementación original (detalladas al final).

1. El guard clause de sesión (línea 21)

```
const { user, logout } = useAuth();
```

```
// Sin sesión (ej: mientras se anima el cierre de sesión) no se dibuja nada
if (!user) return null;
```

Es de las pocas veces que un componente devuelve null en vez de JSX. Es un patrón defensivo: si este componente se renderizara sin un user válido (el comentario aclara el caso típico — el instante en que se ejecuta logout() y AppNavigator todavía no terminó de reconstruir el stack con las nuevas pantallas), evita que el resto del código intente leer user.name, user.email, etc. de null y rompa la app. Devolver null es completamente válido en React: significa “no renderices nada acá”.

2. Sección de perfil

```
<View style={styles.profile}>
  <View style={styles.avatar}>
    <Text style={styles.avatarText}>{user.initials}</Text>
  </View>
  <Text style={styles.name}>{user.name}</Text>
  <Text style={styles.email}>{user.email}</Text>
</View>
```

Vuelca los campos initials, name, email del user que viene de useAuth() — el mismo objeto que AuthContext.js guardó en setUser(found) al hacer login. No hay estado propio ni transformación: es una lectura directa.

3. Las tarjetas de datos — ahora con ProfileDataCard

```
<ProfileDataCard
  title="Datos Personales"
  rows={[
    { icon: '■', label: 'DNI', value: user.dni },
    { icon: '■', label: 'Género', value: user.gender },
    { icon: '■', label: 'Edad', value: user.age },
    { icon: '■', label: 'Peso', value: user.weight },
    { icon: '■', label: 'Altura', value: user.height },
  ]}
/>

<ProfileDataCard
  title="Contacto y Salud"
  rows={[
    { icon: '☎', label: 'Teléfono', value: user.phone },
    { icon: '☎', label: 'Tel. Emergencia', value: user.emergencyPhone },
    { icon: '■', label: 'Obra Social', value: user.healthInsurance },
  ]}
/>
```

```

    { icon: '■', label: 'Contraindicaciones', value: user.contraindications },
  ]}
/>

```

Antes del refactor, esto eran ~110 líneas de JSX: dos bloques `<View style={styles.card}>` con 5 y 4 filas `<View style={styles.dataRow}>` respectivamente, escritas una por una a mano, repitiendo la misma estructura ícono + etiqueta + valor. Ahora cada tarjeta se describe como **datos** (un array de filas) que `ProfileDataCard` recorre con `.map()` para generar el JSX. La pantalla ya no sabe nada de cómo se ve una fila — solo declara qué datos mostrar.

`ProfileDataCard` es un componente de presentación puro (sin hooks, sin lógica): recibe `title` y `rows`, y trae su propio `StyleSheet` interno (`card`, `sectionTitle`, `dataRow`, `icon`, `label`, `value`), así que la pantalla ya no necesita esos estilos definidos localmente. Al ser genérico (no depende de si el perfil es de un entrenador o un alumno), también sirve para `mi_cuenta_user.js`, que tiene la misma estructura de tarjetas.

4. Botón de cerrar sesión — ahora con confirmación

```

// Pide confirmación antes de cerrar sesión, para evitar un toque accidental
const confirmLogout = () => {
  Alert.alert(
    'Cerrar sesión',
    '¿Estás seguro de que querés cerrar sesión?',
    [
      { text: 'Cancelar', style: 'cancel' },
      { text: 'Cerrar sesión', style: 'destructive', onPress: logout },
    ]
  );
};
// ...
<TouchableOpacity style={styles.logoutButton} onPress={confirmLogout}>
  <Text style={styles.logoutText}>Cerrar Sesión</Text>
</TouchableOpacity>

```

Mejora aplicada: originalmente el botón llamaba a `logout` directamente al primer toque, sin ningún aviso. Ahora, tocar “Cerrar Sesión” dispara `Alert.alert` (el diálogo de confirmación nativo del sistema operativo, no un componente propio de la app) en vez de cerrar sesión al instante.

Cómo funciona `Alert.alert(título, mensaje, botones)`:

- Primer argumento: el título del diálogo (“Cerrar sesión”).
- Segundo argumento: el mensaje (“¿Estás seguro de que querés cerrar sesión?”).
- Tercer argumento: un array de botones, cada uno un objeto `{ text, style, onPress }`. El usuario solo puede elegir uno; `Alert.alert` es un componente nativo del sistema operativo, no un Modal propio de la app.
- **`{ text: 'Cancelar', style: 'cancel' }`**: no tiene `onPress` — simplemente cierra el diálogo sin hacer nada más. `style: 'cancel'` es una pista visual para el sistema operativo (en iOS, por ejemplo, este botón se resalta en negrita y queda separado de los demás).
- **`{ text: 'Cerrar sesión', style: 'destructive', onPress: logout }`**: acá recién se llama a la función `logout` real de `AuthContext.js`, y solo si el usuario elige explícitamente esta opción. `style: 'destructive'` pinta el texto en rojo en iOS, marcando visualmente que es una acción irreversible/importante.

Recordá el mecanismo de `logout` que ya se vio en `AuthContext.pdf`:

```
const logout = () => setUser(null);
```

Cuando el usuario confirma: `setUser(null)` → `AuthProvider` se re-renderiza → `AppNavigator` (que consume el contexto) se re-renderiza → como `user` vuelve a ser `null`, el stack cambia de vuelta al grupo “sin sesión” (`login/signup`). El `Alert.alert` es puramente una capa de confirmación **antes** de disparar ese mecanismo — no cambia en nada cómo funciona el `logout` en sí, solo agrega un paso intermedio para evitar un toque accidental. Es el mismo tipo de aviso que ya existe en `editar_rutina_trainer.js` al intentar salir con cambios sin guardar.

5. TrainerBottomNav — ya corregido

```
<TrainerBottomNav
  activeScreen="account"
  isLandscape={isLandscape}
  navigation={navigation}
/>
```

Corrección aplicada: la versión original de esta pantalla no pasaba `isLandscape` a `TrainerBottomNav`, así que en modo horizontal la barra inferior no ajustaba su altura (`bottomNavLandscape`) como sí lo hacen las demás pantallas del proyecto. Ya está corregido: ahora reenvía `isLandscape={isLandscape}` igual que `buscar_usuario_trainer.js` y el resto de las pantallas con barra inferior.

6. Estilos — más cortos tras la limpieza

El `StyleSheet` de este archivo quedó reducido a lo que realmente usa: `container`, `scroll`, `content/contentLandscape`, `header/headerLandscape`, `profile/avatar/avatarText/name/email`, y `logoutButton/logoutText`.

Limpieza aplicada: se eliminaron los estilos `card`, `sectionTitle`, `dataRow`, `icon`, `label`, `value` (ahora viven dentro de `ProfileDataCard.js`, ya no hacen falta acá) y `objective/objectiveIcon/objectiveText`, que eran **código muerto**: no había ningún JSX en este archivo que los usara (eran restos de haber copiado `mi_cuenta_user.js` como base, que sí tiene una sección “Objetivos” con `user.goal`, no presente en el perfil del entrenador).

Resumen para el oral

“mi_cuenta_trainer.js es una pantalla de solo lectura que muestra los datos del user logueado (traído de `useAuth()`) en dos tarjetas armadas con el subcomponente reusable `ProfileDataCard`. El botón de cerrar sesión ahora pide confirmación con `Alert.alert` antes de llamar a `logout`, que dispara el mismo mecanismo de Context que el login: `setUser(null)` hace que `AppNavigator` reconstruya el stack y vuelva a mostrar `login/signup`. Tiene una `guard clause` (`if (!user) return null`) para el instante de transición al cerrar sesión, y ya no arrastra el problema de `isLandscape` faltante ni estilos huérfanos que tenía la versión original.”